

**Instruction-Level Parallelism in Modern Processors: Evolution, Limits, and a Critical  
Review of Recent Research**

Sriman Cherukuru

University of the Cumberland

Computer Architecture

Assignment 4, Part 1: Research Literature Review

September 13, 2026

## **Instruction-Level Parallelism in Modern Processors: Evolution, Limits, and a Critical Review of Recent Research**

Instruction-level parallelism (ILP) is the overlap available among the instructions of one program: independent instructions may execute in the same cycle or out of program order.

Exploiting it has been the main lever for single-thread performance for six decades, and every generation has also rediscovered its limits. This review traces the evolution of ILP, analyzes how processors detect and exploit it and what bounds it, critiques the challenges reported in five recent IEEE and ACM papers published between 2022 and 2024, and synthesizes future directions. It also frames the gem5 experiments of Part 2, which measure pipelining, branch prediction, multiple issue, and multithreading on one simulated core.

### **The Evolution of Instruction-Level Parallelism**

Pipelining came first: overlapping the fetch, decode, execute, memory, and write-back phases of consecutive instructions lets one instruction complete every cycle although each takes several cycles to traverse the machine. Dynamic scheduling came next. Tomasulo (1967) built the floating-point unit of the IBM System/360 Model 91 around reservation stations, a common data bus that forwards results to waiting operands by tag, and an implicit renaming of registers, so independent instructions could bypass a stalled predecessor; the idea remains the core of every out-of-order (OoO) processor. The superscalars of the 1990s added wide fetch, explicit renaming, reorder buffers, speculation, and two-level branch predictors. Wall (1991) supplied the counterpoint: even with very large windows and good prediction, most programs exposed only modest parallelism, and realizable hardware captured a fraction of it. Tullsen, Eggers, and Levy (1995) showed that one thread leaves most issue slots of a wide superscalar empty and proposed simultaneous multithreading (SMT), which lets several threads share one pipeline in the same

cycle. The power wall of the early 2000s ended frequency scaling and turned the industry toward multicore and, later, domain-specific designs, the shift Hennessy and Patterson (2019) call a new golden age. The research reviewed below therefore pursues ILP per watt rather than ILP at any cost.

## **Core Concepts**

### **Parallelism Detection and Exploitation**

Modern cores find parallelism dynamically. Register renaming maps architectural registers onto a larger physical file, removing the write-after-read and write-after-write hazards of register reuse so that only true dependences constrain scheduling. Renamed instructions wait in an instruction queue until their operands and a functional unit are ready, while a reorder buffer (ROB) retires them in program order for precise exceptions and recovery. Control dependences are handled speculatively: a direction predictor, a branch target buffer, and a return address stack choose the next fetch address before the branch resolves, and mispredicted work is squashed. Multiple issue widens every stage, and SMT fills the slots one thread leaves idle with another thread's ready instructions.

### **Fundamental Limitations**

Three classes of limits bound ILP. Data dependences form chains whose latencies add: a loop that updates one variable through a multiply and an add cannot run faster than one iteration per multiply-plus-add latency however wide the machine is, and dependent cache misses serialize in the same way. Control dependences cost a pipeline refill per misprediction, and the cost grows with depth and width because more speculative work is discarded. Resources form the third class: window size, rename bandwidth, wakeup logic, and register and cache ports grow super-linearly with width, and memory latencies of hundreds of cycles fill even large windows.

## Performance Metrics and Trade-offs

Instructions per cycle (IPC), or its inverse CPI, is the canonical metric because execution time is instruction count times CPI times cycle time. Throughput and latency diverge in an OoO core: IPC measures throughput, but each instruction's fetch-to-commit latency grows with the number of instructions in flight. By Little's law, latency equals window occupancy divided by throughput, so a wider window raises latency even as it raises throughput. Branch quality is expressed as a misprediction rate or as mispredictions per thousand instructions (MPKI), and speculation waste as the fraction of fetched instructions that are squashed. Because power scales with speculative and bookkeeping work, energy per instruction and the energy-delay product are the honest yardsticks.

## Current Challenges and How Research Addresses Them

Koizumi et al. (2023) attack the cost of renaming itself. Naming operands by register number forces an OoO core to rename every instruction, and the multi-ported map tables and checkpoints this requires consume power and cap front-end width. Their Clockhands instruction set specifies a source operand as “the value written in this register group  $k$  times before” (p. 1), so no renaming is needed, while keeping instruction counts near a conventional RISC, unlike the earlier rename-free STRAIGHT design, which inflated code by roughly a third. They report 7.4% lower energy than RISC at comparable performance on an eight-fetch core and 24.4% lower on a sixteen-fetch core.

Khan et al. (2022) find that even the state-of-the-art TAGE-SC-L predictor leaves an average of 3.0 mispredictions per thousand instructions on data-center applications because their code footprints exhaust its tables. Whisper profiles production runs, isolates the contexts that mispredict, and encodes each hard branch's correlation with earlier outcomes as compact

Boolean formulas embedded in the binary. It removes 16.8% of the remaining mispredictions for a 2.8% average speedup, 7.9% better than earlier profile-guided schemes. The gain is modest, and part of the prediction problem moves into a profiling pipeline.

Shukla et al. (2022) argue that the memory wall is a series of latency walls, one per cache level: a level-one hit is roughly forty times faster than DRAM, yet removing its latency offers a comparable opportunity because hits are so frequent. Register File Prefetching uses the existing scheduler and spare bandwidth to deliver 43.4% of load results into the register file before the loads issue, gaining 3.1% on a Tiger-Lake-class core and 5.7% on an up-scaled core. At the far end of the hierarchy, Naithani et al. (2023) target indirect memory-access chains that defeat prefetchers and fill the ROB with stalled loads. Decoupled Vector Runahead runs a lightweight speculative sub-thread that vectorizes the dependent load chain of many future iterations and issues those loads early; with 1,139 bytes of state it reaches 2.4 times the performance of a five-wide core with a 350-entry ROB and twice that of Vector Runahead.

Bera et al. (2024) observe that loads constrain ILP twice: consumers wait for their data, and the loads themselves occupy queue entries, cache ports, and scheduler slots, which value prediction leaves untouched. Constable identifies likely-stable loads that keep returning the same value from the same address, tracks writes to their sources, and stops executing them until something changes. It improves performance by 5.1% and cuts core dynamic power by 3.4%, and the gain rises to 8.8% with two-way SMT, so ILP and thread-level techniques interact.

Read together, the papers show that gains from general-purpose techniques sit in the low single digits, 2.8% to 5.1%, each bought with new predictors, tables, and verification effort, whereas order-of-magnitude gains appear only where a technique targets one behavior such as indirect chains. Every technique adds speculative state that, after Spectre, must be examined for

side channels, and all are evaluated in simulation against differing baselines, one reason open simulators such as gem5 (Binkert et al., 2011) matter.

### **Synthesis of Future Directions**

Five directions emerge. First, instruction-set and microarchitecture co-design: Clockhands shows that the ISA can shed renaming and afford much wider front ends. Second, decoupled speculation: helper sub-threads and decoupled front ends pursue memory- and instruction-level parallelism outside the ROB instead of enlarging monolithic windows. Third, learned prediction: Whisper’s formulas and machine-learning predictors treat prediction as a learning problem constrained by hardware cost. Fourth, dependence breaking through stability: Constable and Register File Prefetching exploit predictable values and addresses, and their synergy with SMT suggests that future cores will layer several such mechanisms. Fifth, heterogeneity: as Hennessy and Patterson (2019) argue, accelerators and asymmetric cores absorb throughput work, so the ILP question becomes how large a general-purpose core should be for the irregular code that remains, which Koizumi et al. (2023) note only OoO execution speeds up.

### **Conclusion**

ILP research has moved from discovering parallelism to affording it. The classical machinery remains in every high-performance core, but each additional percent of IPC now costs power and complexity. The recent literature answers with rename-free instruction sets, profile-guided predictors, register-file prefetching, decoupled runahead, and load elimination. Part 2 of this assignment tests the classical side of this story in gem5.

## References

- Bera, R., Ranganathan, A., Rakshit, J., Mahto, S., Nori, A. V., Gaur, J., Olgun, A., Kanellopoulos, K., Sadrosadati, M., Subramoney, S., & Mutlu, O. (2024). Constable: Improving performance and power efficiency by safely eliminating load instruction execution. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)* (pp. 88–102). IEEE. <https://doi.org/10.1109/ISCA59077.2024.00017>
- Binkert, N., Beckmann, B., Black, G., Reinhardt, S. K., Saidi, A., Basu, A., Hestness, J., Hower, D. R., Krishna, T., Sardashti, S., Sen, R., Sewell, K., Shoaib, M., Vaish, N., Hill, M. D., & Wood, D. A. (2011). The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2), 1–7. <https://doi.org/10.1145/2024716.2024718>
- Hennessy, J. L., & Patterson, D. A. (2019). A new golden age for computer architecture. *Communications of the ACM*, 62(2), 48–60. <https://doi.org/10.1145/3282307>
- Khan, T. A., Ugur, M., Nathella, K., Sunwoo, D., Litz, H., Jiménez, D. A., & Kasikci, B. (2022). Whisper: Profile-guided branch misprediction elimination for data center applications. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)* (pp. 19–34). IEEE. <https://doi.org/10.1109/MICRO56248.2022.00017>
- Koizumi, T., Shioya, R., Sugita, S., Amano, T., Degawa, Y., Kadomoto, J., Irie, H., & Sakai, S. (2023). Clockhands: Rename-free instruction set architecture for out-of-order processors. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)* (pp. 1–16). ACM. <https://doi.org/10.1145/3613424.3614272>
- Naithani, A., Roelandts, J., Ainsworth, S., Jones, T. M., & Eeckhout, L. (2023). Decoupled vector runahead. In *Proceedings of the 56th Annual IEEE/ACM International Symposium*

*on Microarchitecture (MICRO '23)* (pp. 17–31). ACM.

<https://doi.org/10.1145/3613424.3614255>

Shukla, S., Bandishte, S., Gaur, J., & Subramoney, S. (2022). Register file prefetching. In

*Proceedings of the 49th Annual International Symposium on Computer Architecture*

*(ISCA '22)* (pp. 410–423). ACM. <https://doi.org/10.1145/3470496.3527398>

Tomasulo, R. M. (1967). An efficient algorithm for exploiting multiple arithmetic units. *IBM*

*Journal of Research and Development*, 11(1), 25–33. <https://doi.org/10.1147/rd.111.0025>

Tullsen, D. M., Eggers, S. J., & Levy, H. M. (1995). Simultaneous multithreading: Maximizing

on-chip parallelism. In *Proceedings of the 22nd Annual International Symposium on*

*Computer Architecture (ISCA '95)* (pp. 392–403). ACM.

<https://doi.org/10.1145/223982.224449>

Wall, D. W. (1991). Limits of instruction-level parallelism. In *Proceedings of the Fourth*

*International Conference on Architectural Support for Programming Languages and*

*Operating Systems (ASPLOS IV)* (pp. 176–188). ACM.

<https://doi.org/10.1145/106972.106991>